Inventory

Project overview, what it is made of, and how the Vue frontend connects

Project	Inventory - veterinary-pharmaceutical distribution (Egypt / MENA)
Document	Client overview and technical summary
Version / date	October 2026
What this covers	The complete back end (API, business rules, accounting) and the contract the Vue web application builds on

1. In one paragraph

Inventory is a complete business-management system for a company that **buys medicines from suppliers, stores them in several warehouses (some cold-storage), and sells them on credit or cash** to clinics, farms, poultry operations and distributors. It handles the whole chain - purchasing, stock with expiry control, sales and invoicing, customer management, accounting, expenses, stock write-offs and in-house manufacturing - in one system, with every action recorded and every money movement posted to the books automatically. The back end is a secure API; the **Vue web application** is the screen layer on top of it that your staff use.

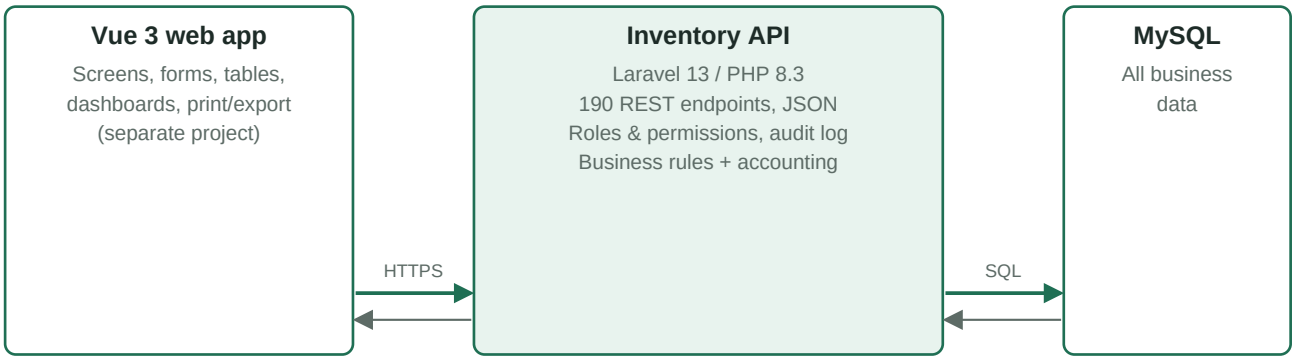

Figure 1 - how the pieces fit. The Vue app never touches the database: every screen reads and writes through the API, which enforces the rules, permissions and accounting.

2. What the system does

Module	What your team can do
Inventory	Warehouses, products and categories; stock kept in batches with expiry dates and sold **first-expired-first-out (FEFO)**; transfers between warehouses; goods receipt; **packaging units** (e.g. packet → package → carton) per product; free-gift stock; availability of a product in every warehouse.
Sales	Sales orders (cash or credit, with credit-limit checks), invoices, deliveries, customer payments (collections) and returns. One order can draw from **several warehouses**; a sale and its delivery can be done in **one step**; each product has a **minimum selling price** that sellers cannot go below; free-gift lines at price 0.
Purchasing	Suppliers (with address and contract file), purchase orders, receiving stock (including **free gifts** from the supplier), supplier invoices. Buyers raise orders with quantities only; an **administrator sets the cost** in a "pending pricing" step.
CRM	Customers, leads, visit log, complaints, marketing campaigns - with sales reps seeing only their own customers.
Accounting	Chart of accounts, automatic double-entry journal entries from every sale, purchase, collection, expense and write-off; balance sheet, income statement, receivable and payable ageing.
Expenses	Record operating expenses with receipt files; approval by an accountant posts them to the books.
Recap (write-offs)	Remove expired / damaged / lost stock. Approval deducts the stock **and posts the loss to the chart of accounts**; expired stock can be drafted automatically every day.
Manufacturing	Bills of materials and production orders: consumes ingredients (FEFO), creates the finished-goods batch at its true cost and posts the cost to accounts.
Analytics	Dashboard KPIs, expiry tracking, stock roll-up, sales and inventory reports, and a **product life-cycle** view from purchase, through transfers, to sale.
Administration	Users, **roles and permissions the admin builds** (including who may edit or delete), tamper-evident audit log, notifications, Excel / CSV import and export.

3. Features that matter to the business

- **Expiry safety.** Stock is always sold earliest-expiry first; expired stock is flagged and can be written off in a controlled, approved step.
- **Accounting without double entry.** Sales, purchases, payments, expenses, write-offs and production post balanced journal entries automatically; nothing is typed twice.
- **Price protection.** Product cost is visible to administrators only; sellers see the selling price and a minimum price floor that the system enforces on every order line.
- **Approval workflows.** Purchase pricing, expense approval, write-off approval and production completion each need the right permission.
- **Full control of access.** Ten ready-made roles, plus roles the admin can create and tune permission by permission (view / add / edit / delete / approve / print / export / audit).
- **Safe editing and deleting.** Records can be edited or deleted only by those allowed; anything already used elsewhere is protected; posted financial documents are corrected by reversal (for example voiding an invoice), never overwritten.
- **Bulk loading and reporting.** Import customers, products, suppliers, warehouses and invoices from Excel; export any list to Excel or CSV.
- **Tamper-evident audit trail.** Every change is logged with who, when and what; the log is hash-chained so any later tampering is detectable.

4. How the Vue frontend connects

Important: the Vue application is a **separate project** from this back end - they are developed, built and deployed independently and talk only through the API. This section is the contract the Vue app builds on; everything in it is already implemented and tested on the back end.

4.1 Access and permissions

- Every request from the Vue app carries an **access token** in the header `Authorization: Bearer <token>`. No cookies and no CSRF handling are needed.
- The API tells the app the user's **permissions list** (strings such as `sales.edit`). The Vue app shows or hides menus, buttons and screens by checking these strings - it never hard-codes role names. Because admins can create and change roles, the screens adapt automatically.
- `GET /api/v1/auth/me` returns the current user and permissions, so the app can restore its state after a page refresh.

4.2 Typical call from the Vue app

```
import axios from 'axios'

const api = axios.create({ baseURL: import.meta.env.VITE_API_URL + '/api/v1' })
api.interceptors.request.use(c => { c.headers.Authorization = `Bearer ${auth.token}`; return c })

// list with search, filter, sort and paging - same options on every list
const { data } = await api.get('/products', {
  params: { search: 'amox', 'filter[active]': true, sort_by: 'name', sort_dir: 'asc', page: 1 }
})
// -> { data: [...], links: {...}, meta: { current_page, last_page, total } }

// choose the warehouses to sell a product from, then create an order
await api.get(`/products/${id}/availability`)
await api.post('/sales-orders', { customer_id, warehouse_id, pay_type: 'cash', lines })
```

4.3 Conventions the frontend relies on

Topic	Behaviour
Responses	JSON everywhere. Lists come as `{ data, links, meta }` (15 per page by default, `per_page` to change); single records as `{ data }`.
Validation errors	**422** with `{ message, errors: { field: [messages] } }` - maps directly onto form fields, including nested ones such as `lines.0.unit_price`.
Other statuses	**401** missing or expired token · **403** not allowed · **404** not found / other company · **409** cannot delete, still in use · **204** deleted.
Search, filter, sort	Every list accepts `search`, `filter[column][op]`, `sort_by`, `sort_dir` - one table component serves every screen.
Export	`?export=csv|xlsx` on any list returns a file (all matching rows) - use a blob download.
File upload	Two steps: upload the file (multipart) to get a path, then send that path with the record (contracts, receipts, supplier and delivery invoices).
Import	Multipart upload of an Excel / CSV; a downloadable template per screen; a row-by-row error report if anything is wrong (nothing is half-imported).
CORS	In production the Vue app's web address is set once in the server setting `FRONTEND_URLS`; other sites are refused.

4.4 Suggested screens and the endpoints behind them

Area	Screens	Endpoints (examples)
Sales	Orders list and form (multi-warehouse), quick sale + delivery, invoices, collections, returns	sales-orders, sales-orders/deliver, invoices, collections, returns
Inventory	Products (with packaging units), warehouses, batches, transfers, goods receipt, product life cycle	products, products/{id}/availability, products/{id}/lifecycle, transfers
Purchasing	Suppliers, purchase orders, admin pricing queue, receiving with free gifts	purchase-orders, purchase-orders/pending-pricing, .../pricing, .../receive
CRM	Customers, leads, visits, complaints, campaigns	customers, leads, visits, complaints, campaigns
Accounting	Chart of accounts, journal, balance sheet, income statement, ageing	chart-of-accounts, journal-entries, reports/*
Operations	Expenses, write-offs (recap), BOMs, production orders	expenses, recaps, boms, production-orders
Admin	Users, roles and permissions, units, audit log, notifications	users, roles, permissions, units, audit-log, notifications

5. What it is made of

Layer	Technology	Why
API / business logic	Laravel 13, PHP 8.3, organised as 10 independent modules (Core, Inventory, Sales, Purchasing, CRM, Accounting, Expenses, Recap, Manufacturing, Analytics)	Mature, secure framework; modules keep each business area separate and maintainable
Database	MySQL	Reliable transactions - stock, invoices and journal entries are saved together or not at all
API tokens	Laravel Sanctum (bearer tokens)	Simple, safe token access that suits a single-page Vue app
Roles & permissions	Spatie Laravel-Permission	Admin-editable roles with fine-grained permissions
Excel / CSV	Maatwebsite Excel	Real .xlsx import and export
Frontend (separate project)	Vue 3 + Vite, Vue Router, Pinia, Axios (recommended)	Consumes the API above; any Vue 3 setup works since the contract is plain REST / JSON
Quality	PHPUnit - 598 automated tests, code-style checks	Business rules, permissions and accounting are covered by tests
Documentation	API reference (one page per module), Postman collection with 224 requests	Lets frontend developers and testers work without reading code

6. Security and data integrity

- **Each company's data is isolated**; a user can never read or change another company's records.
- **Every action is permission-checked on the server** - hiding a button in Vue is a convenience, not the protection.
- Passwords are stored hashed; API tokens can be revoked; requests are rate-limited.
- Money and stock changes run as **atomic transactions**; if any step fails nothing is saved.
- The audit log is **hash-chained** and verifiable (an endpoint checks its integrity).
- Uploaded files are accepted only from the platform's own upload step, by type and size.

7. Delivery status

Item	Status
Back end API (190 endpoints across 10 modules)	Complete and tested
Business rules: FEFO, credit limits, minimum prices, approvals, automatic accounting	Complete and tested
Automated tests	598 passing
API documentation and Postman collection	Complete; the newest requests are flagged for review
Vue web application	Separate project - built against this API (not part of the back-end repository)
Settings screen	Not built - no data behind it in the specification
Production hosting, HTTPS, backups	To be set up at go-live (see below)

8. Going live - checklist

- Server with PHP 8.3, MySQL and a web server over **HTTPS**; run the database migrations and the starter data (roles, chart of accounts, units).
- Set the production settings: database, application key, audit key, and `FRONTEND_URLS` with the Vue app's address.
- Enable the scheduler (daily jobs: overdue invoices, expiry alerts, expired-stock write-off drafts).
- Build the Vue app (`VITE_API_URL` pointing at the API) and serve it as static files.
- Create the first administrator, then build the company's roles and users from the Roles screen; load existing customers, products, suppliers, warehouses and invoices with the Excel import.
- Back up the database daily and test a restore.

Appendix - ready-made roles

Role	Intended use
Administrator	Full access, including delete, pricing of purchase orders and role management
Owner	Oversight: view, approve, print, export, audit across every module
Sales Manager / Sales Rep	Sales module; the rep sees only their own customers and orders
Warehouse Manager / Employee	Inventory, write-offs and manufacturing (employee cannot approve)
Accountant	Accounting and expenses, expense approval
Purchasing	Suppliers and purchase orders (costs are set by the administrator)
Customer Service	CRM: customers, visits, complaints
Auditor	Read, export and audit across the whole system

Roles are not fixed: the administrator can create new roles and change any role's permissions at any time.